

Hive API Gateway

Developer Reference | v0.1.0

Host: 10.10.0.200 • Port: 1717 • ROS 2 Humble

Overview

The Hive API Gateway is a FastAPI-based HTTP server that bridges external clients to a ROS 2 robot system. It runs on port 1717 and exposes endpoints for robot task dispatching, map image serving, coordinate conversion, and lifecycle management of SLAM and navigation nodes.

Base URL	http://10.10.0.200:1717
Framework	FastAPI + Uvicorn
ROS 2	Humble (Ubuntu 22.04)
Auth	None — internal network only
Format	JSON (request & response)
Version	0.1.0

Endpoint Summary

Method	Endpoint	Description
GET	/health	ROS readiness check
POST	/tasks	Dispatch a robot behavior / navigation task
GET	/map/meta	Map metadata and coordinate bounds
POST	/map/preview	Dry-run pixel-to-map coordinate conversion
GET	/map_image	Serve a .pgm map file rendered as PNG
GET	/map_image/list	List all available .pgm map files
GET	/lifecycle_status	Live SLAM and NAV status
POST	/lifecycle_start	Start SLAM and localization nodes
POST	/lifecycle_stop	Stop and clean up localization nodes

GET /health

Returns whether the ROS 2 node is initialised and ready to accept tasks. Use this as a liveness probe before sending any robot commands.

Response

```
{
  "ros_ready": true
}
```

Example

```
curl http://10.10.0.200:1717/health
```

POST /tasks

Dispatches a behavior or navigation task to the Hive robot. Accepts waypoints in three formats — the gateway auto-detects and converts pixel coordinates to ROS map-frame metres when needed.

Input Formats

Format	Key Field	Notes
1. Pixel waypoints	<code>pixel_waypoints[]</code>	Preferred. Each item: col, row, yaw_deg.
2. Poses (pixels)	<code>poses[]</code>	Auto-detected when coords fall outside valid map bounds.
3. Poses (map metres)	<code>poses[]</code>	Real ROS map-frame coordinates, passed through unchanged.

Example Request

```
curl -X POST http://10.10.0.200:1717/tasks \
-H "Content-Type: application/json" \
-d '{
  "id": 22,
  "behavior_name": "FollowRoute",
  "pixel_waypoints": [
    { "col": 254, "row": 389, "yaw_deg": -3 },
    { "col": 310, "row": 420, "yaw_deg": 0 }
  ]
}'
```

Response

```
{
  "accepted":      true,
  "task_id":       "a3f1c2d4-...",
  "behavior":      "FollowRoute",
  "waypoint_count": 2,
  "source":        "pixel_waypoints"
}
```

GET /map/meta

Returns the active map metadata and valid coordinate bounds. Use this to understand the pixel-to-metre conversion parameters before sending waypoints.

Response

```
{
  "resolution": 0.05,
  "origin_x":   -9.9,
  "origin_y":   -14.85,
  "img_w_px":   513,
  "img_h_px":   706,
  "frame_id":   "map",
  "bounds": {
    "x": [-9.9, 15.75],
    "y": [-14.85, 20.45]
  }
}
```

Coordinate conversion formula:

```
map_x = origin_x + col * resolution
map_y = origin_y + (img_h - 1 - row) * resolution

// Y is flipped: row 0 (top of image) = maximum map Y
```

POST /map/preview

Dry-run endpoint that converts pixel waypoints to ROS map-frame poses without dispatching any command to the robot. Use this to verify coordinate conversion before sending real tasks.

Example

```
curl -X POST http://10.10.0.200:1717/map/preview \
```

```
-H "Content-Type: application/json" \  
-d '{"pixel_waypoints": [{"col": 254, "row": 389, "yaw_deg": -3}]}'
```

Response

```
{  
  "poses": [{  
    "header": { "frame_id": "map" },  
    "pose": {  
      "position": { "x": 2.8, "y": 0.95, "z": 0.0 },  
      "orientation": { "x": 0.0, "y": 0.0, "z": -0.026, "w": 0.999 }  
    }  
  }],  
  "source": "pixel_waypoints",  
  "map_meta": { ... }  
}
```

GET /map_image

Serves a .pgm map file from the package resource folder, converted to PNG so any browser or HTTP client can render it directly. Defaults to new_office_map.pgm if no file parameter is specified.

Query param	file (optional) — filename of the .pgm to serve
Default	new_office_map.pgm
Response	image/png — rendered inline in browser

Examples

```
# Default map — open in browser  
http://10.10.0.200:1717/map_image  
  
# Specific map file  
http://10.10.0.200:1717/map_image?file=other_map.pgm  
  
# Download via curl  
curl http://10.10.0.200:1717/map_image --output map.png
```

GET /map_image/list

Lists all .pgm files available in the package resource folder.

```
{  
  "maps": ["new_office_map.pgm", "warehouse.pgm"],  
}
```

```
"count": 2
}
```

To add a new map: drop the .pgm file into resource/, declare it in setup.py under data_files, then rebuild with colcon build.

GET /lifecycle_status

Returns the live status of the SLAM and navigation systems. Both checks run concurrently so the response is never slower than the slower of the two checks (~2 seconds worst case).

SLAM_status: ON	global_localization_lifecycle_node state is 'active'
SLAM_status: OFF	Node is 'unconfigured', 'inactive', or unreachable
NAV_status: ON	/navigate_to_pose action server responds within 2s
NAV_status: OFF	Action server timeout or unavailable

Response

```
{
  "SLAM_status": "ON",
  "NAV_status": "ON"
}
```

Example

```
curl http://10.10.0.200:1717/lifecycle_status
```

POST /lifecycle_start

Runs a full SLAM and localization startup sequence across two ROS 2 lifecycle nodes. Each step is executed in order — if any step fails the endpoint returns 500 immediately with the failed step details.

Startup Sequence

#	ROS 2 Command	Notes
1	<code>lifecycle set /auto_localizer_lifecycle_node activate</code>	Direct activate attempt
↳	<code>lifecycle set /auto_localizer_lifecycle_node deactivate</code>	Only if step 1 fails
↳	<code>lifecycle set /auto_localizer_lifecycle_node</code>	Retry after deactivate

#	ROS 2 Command	Notes
	activate	
2	lifecycle set /global_localization_lifecycle_node configure	Configure global localizer
3	lifecycle set /global_localization_lifecycle_node activate	Activate global localizer

Response (success)

```
{
  "accepted": true,
  "action": "lifecycle_start",
  "steps": [
    { "node": "/auto_localizer_lifecycle_node", "transition": "activate", "ok": true },
    { "node": "/global_localization_lifecycle_node", "transition": "configure", "ok": true },
    { "node": "/global_localization_lifecycle_node", "transition": "activate", "ok": true }
  ]
}
```

Example

```
curl -X POST http://10.10.0.200:1717/lifecycle_start
```

POST /lifecycle_stop

Gracefully stops the global localization node by running deactivate followed by cleanup. Deactivate must succeed before cleanup is attempted.

Shutdown Sequence

#	ROS 2 Command	Notes
1	lifecycle set /global_localization_lifecycle_node deactivate	Deactivate global localizer
2	lifecycle set /global_localization_lifecycle_node cleanup	Release resources

Response (success)

```
{
  "accepted": true,
```

```

    "action": "lifecycle_stop",
    "steps": [
      { "node": "/global_localization_lifecycle_node", "transition": "deactivate", "ok": true },
      { "node": "/global_localization_lifecycle_node", "transition": "cleanup", "ok": true }
    ]
  }
}

```

Example

```
curl -X POST http://10.10.0.200:1717/lifecycle_stop
```

Error Reference

Code	Meaning	Common Cause
400	Bad request	Invalid filename format on /map_image
404	Not found	Requested .pgm file does not exist in resource folder
422	Validation error	Missing col/row in pixel_waypoints
500	Server error	ROS goal rejected, lifecycle transition failed
503	Service unavailable	ROS not initialised or Hive action server unreachable

Notes for Future Development

- `lifecycle_stop` — `auto_localizer_lifecycle_node deactivate` is not yet wired — add when shutdown sequence is confirmed
- `lifecycle_status` — `SLAM_status` uses a CLI subprocess call (`ros2 lifecycle get`). A cleaner approach would be a native `rcply` service client to avoid spawning a child process on every poll
- `NAV_status check` — Uses `wait_for_server` with a 2s timeout. If the nav stack takes longer to come up, increase this timeout or cache the last known state
- `Map metadata` — `MAP_META` is hardcoded. Consider loading it from the `.yaml` sidecar file alongside the `.pgm` at startup to avoid drift when maps are updated
- `Task IDs` — `task_id` is a UUID generated server-side if not supplied. Clients should supply their own for traceability
- `Behavior ID 21` — Has special handling — populates `goal.pose` from the first waypoint. Document or refactor this once the behavior interface is stable
- `CORS` — Currently allows all origins (*). Restrict to known webapp origins in production
- `Pillow dependency` — Required for `.pgm` to `PNG` conversion in `/map_image`. Ensure it is pinned in requirements or the Dockerfile

Quick Reference Card

Health

```
curl http://10.10.0.200:1717/health

# Send task with pixel waypoints
curl -X POST http://10.10.0.200:1717/tasks -H "Content-Type: application/json" \
  -d '{"id":22,"behavior_name":"FollowRoute","pixel_waypoints":
[{"col":254,"row":389,"yaw_deg":-3}]}'

# Map metadata
curl http://10.10.0.200:1717/map/meta

# Preview coordinate conversion
curl -X POST http://10.10.0.200:1717/map/preview -H "Content-Type: application/json" \
  -d '{"pixel_waypoints":[{"col":254,"row":389,"yaw_deg":-3}]}'

# View map in browser
http://10.10.0.200:1717/map_image

# List available maps
curl http://10.10.0.200:1717/map_image/list

# Lifecycle status
curl http://10.10.0.200:1717/lifecycle_status

# Start SLAM + localization
curl -X POST http://10.10.0.200:1717/lifecycle_start

# Stop localization
curl -X POST http://10.10.0.200:1717/lifecycle_stop
```